Vienna Vulkan Engine

Eight pages to your first visible program.

VVEngine opens a window, loads scenes and draws them with Vulkan. Your program uses the public **vve** API. This quickstart uses Windows, MSVC and the simple implementation, matching your current setup.

1. Install the tools

Install Git, Visual Studio with **Desktop development with C++** and **C++ CMake tools** (Ninja), CMake 3.31.8 or newer, and the Vulkan SDK. Reopen PowerShell after installing. The SDK supplies VULKAN_SDK. The renderer requests a Vulkan 1.4 instance and checks required Vulkan 1.3 GPU features.

2. Clone the source and install vcpkg

The commands below assume C:/data/GitHub exists. Use another parent directory if needed. If a folder already exists, use it instead of cloning again.

```
cd C:\data\GitHub
git clone https://github.com/hlavacs/ViennaVulkanEngine.git
git clone https://github.com/microsoft/vcpkg.git
cd vcpkg
.\bootstrap-vcpkg.bat
$env:VCPKG_ROOT = 'C:\data\GitHub\vcpkg'
cd ..\ViennaVulkanEngine
```

> **What the build tools do**
> CMake configures targets and dependencies. Ninja executes build commands. MSVC compiles C++. vcpkg installs third-party libraries. Vulkan and Slang come from the SDK.

Read in order: **2** build/run, **3** link your own program, **4** API, **5-6** examples, **7** assets, **8** troubleshooting. Keep the full manual for implementation details.

Sources: R, C | source key on page 8

2. Compile and run the examples

Start with the supplied program before changing code.

```
# Run from ViennaVulkanEngine in ordinary PowerShell.
.\build_windows.cmd debug
.\bin\debug\exe\game.exe --frames 120
```

The script initializes the Visual Studio compiler, installs vcpkg dependencies, configures with Ninja, builds and runs tests. The first build can take much longer than later builds. --frames limits the supplied game example; omit it to play until the window closes.

Program	What it demonstrates
game.exe	Repeated grass plane, crates, camera, score and V2 music controls.
audio_demo.exe	Minimal public audio API demonstration.
testscene.exe / sponza.exe	Imported scenes; sponza uses a camera fitted to its scale.
postprocessing.exe	Image effects and interactive settings.
hierarchy_car.exe	A car built from parts; moving its root and wheel nodes moves every part below them.

Rebuild and select a build type

```
.\build_windows.cmd debug
.\build_windows.cmd release
# Build only, without the post-build tests:
.\build_windows.cmd debug --no-tests
```

Debug output is under bin/debug/exe; release output is under bin/release/exe. Run from the repository root so relative asset paths resolve. A successful compile followed by a failed test is a separate test failure; read the named CTest failure.

Debug in VS Code

Open the repository folder. Install the Microsoft C/C++ and CMake Tools extensions. Choose a Visual Studio amd64 kit and the Debug variant. Select an executable launch entry that points to bin/debug/exe. Set the working directory to the repository root. Build and place a breakpoint in main before starting the debugger.

Sources: R, E, H | source key on page 8

3. Link your own program

The simplest first project lives inside the example build.

Create **examples/mygame/main.cpp** and paste the complete program from page 5. Create **examples/mygame/CMakeLists.txt** containing:

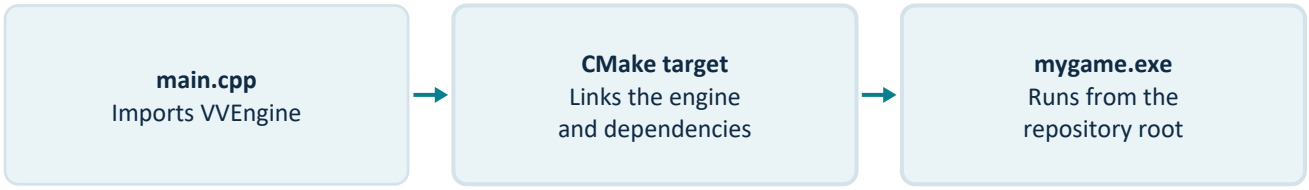
```
add_executable(mygame main.cpp)
target_link_libraries(mygame PRIVATE
    ViennaVulkanEngine::ViennaVulkanEngine)
vve_set_output_dirs(mygame)
```

At the bottom of **examples/CMakeLists.txt**, add this registration line once:

```
add_subdirectory(mygame)
```

Then rebuild from the engine repository root:

```
.\build_windows.cmd debug --no-tests
.\bin\debug\exe\mygame.exe
```



import VVEngine; makes API declarations available. **target_link_libraries** links the implementation and propagates the C++23/module settings and dependencies. You need both. Do not include Engine.ixx as a header or manually copy compiler-specific module files.

This recipe inherits the engine's existing import std setup. A separate parent CMake project also needs its CMake-version-specific import std gate set before project(), plus CMAKE_CXX_MODULE_STD=ON. Stay with this example layout for a first program; the README describes that advanced integration.

> **One engine target**
> Link ViennaVulkanEngine::ViennaVulkanEngine. Add vve_example_support only if you import VVE.ExampleSupport. Do not separately link SDL/ImGui archives just to use the facade.

Sources: C, E, R | source key on page 8

4. The API you need first

Objects and handles in simple language.

API	Use it for
Engine / EngineBuilder	Own the engine; choose windows; init, run or step.
engine.world().get<T>()	Access a subsystem of this engine.
RenderSystem	Create visible objects, set camera/lights and update transforms.
AssetSystem	Import a file into an asset scene.
WindowSystem / InputState	Read keys and window state.
GuiSystem	draw builds UI; configureFonts sets fonts before the first frame.
AudioSystem	Initialize audio, load sounds, play voices and control gain.
ECS	Store game entities and components.
Transform / Camera / handles	Describe placement/view; retain typed resource identities.

Creation and frames

EngineBuilder constructs the owner. init starts the platform and user hooks; audio.init opens sound output separately. run calls init and repeats step until stopping. step polls input, updates your systems and draws. Use run for a simple loop, or step for your own outer loop.

Check fallible calls

```
auto result = render.addPlane(
    vve::Vec2{10,10}, vve::LinearColor{});
if (!result) {
    std::cerr << vve::errorName(result.error()) << '\n';
    return 1;
}
vve::RenderObjectHandle floor = *result;
```

A std::expected contains either a value or an Error. Check it before dereferencing. Wrapper/world views borrow the engine's data; keep the engine alive while using them. Use the facade namespace vve, without calling vve::simple internals.

Sources: A, T, W, G | source key on page 8

5. Example: a window and a scene

Complete main.cpp for the target on page 3.

```
import std;
import VVEngine;

int main() {
    auto engine = vve::EngineBuilder<>{}
        .applicationName("My first game")
        .addWindow(vve::WindowSetup{}
            .title("Hello VVEngine")
            .extent(vve::PixelExtent{800, 600}))
        .build();
    if (auto r = engine.init(); !r) {
        std::cerr << vve::errorName(r.error()) << '\n';
        return 1;
    }
    auto& render = engine.world().get<vve::RenderSystem>();
    if (auto r = render.loadSampleScene(); !r) return 2;
    if (auto r = engine.run(); !r) {
        std::cerr << vve::errorName(r.error()) << '\n';
        return 3;
    }
    return 0;
}
```

Build and launch mygame as shown on page 3. A window appears with the engine's sample scene. Closing it stops run. The error branches report initialization or frame failures; scene loading returns code 2 if it fails. The scene is built in, so the program needs no model path. engine.world returns access to the same renderer owned by this engine. The reference named render changes that renderer; it is not a separate graphics engine.

Start here
Make this program run before adding your own model, physics or game rules. Keep main.cpp small and move ongoing game behavior into a user system when needed.

Sources: A, D, W | source key on page 8

6. Example: your game update

Replace main.cpp with this complete user-system example.

```
import std;
import VVEngine;

// Engine owns this system and calls these two hooks.
struct Game {
    double elapsed = 0;
    auto init(auto& world) -> std::expected<void, vve::Error> {
        return world.template get<vve::RenderSystem>()
            .loadSampleScene();
    }
    void update(auto& world, const vve::FrameContext& frame) {
        elapsed += frame.delta_time.seconds;
        auto& windows = world.template get<vve::WindowSystem>();
        if (windows.input().wasKeyPressed(vve::Key::p))
            std::cout << "P at " << elapsed << " s\n";
    }
};

int main() {
    vve::Engine<Game> engine;
    if (auto r = engine.run(); !r) {
        std::cerr << vve::errorName(r.error()) << '\n';
        return 1;
    }
}
```

Game::init loads the scene once. Game::update runs each frame after input polling and before drawing. The delta is in seconds. Press P to print the elapsed time. The engine owns a Game value and invokes these hooks through the public API.

The template keyword is required after world because its type is generic in these functions. Use elapsed time for movement, for example distance = speed * frame.delta_time.seconds. Additional user systems run in the declared order.

Sources: A, W, G, T | source key on page 8

7. Models, textures and music

Load data, place it, then add sound through the facade.

These excerpts belong after engine.init. modelPath, texturePath and musicPath must name existing files; use absolute paths or a defined asset directory.

```
auto& assets = engine.world().get<vve::AssetSystem>();
auto& render = engine.world().get<vve::RenderSystem>();
auto scene = assets.loadScene(modelPath);
if (!scene) return 4;
auto instance = render.instantiateScene(*scene);
if (!instance) return 5;
```

loadScene stores imported data; instantiateScene creates visible placements. Choose a suitable camera and light. Camera::clip defaults to near 0.1 and far 100; fit these distances to a large level.

```
auto floor = render.addTexturedPlane(
    vve::Vec2{20,20}, texturePath, vve::Vec2{10,10});
if (!floor) return 6;
```

One 40 by 40 XZ plane has UVs spanning 0..10. Repeat addressing tiles the image. Retain handles to move, hide or remove visible objects.

Move a model with its parts

```
auto root = assets.sceneRootNode(*scene);
auto car = render.instantiateScene(*scene,
    vve::SceneInstantiationOptions{.keep_hierarchy = true});
if (!root || !car) return 10;
auto local = assets.nodeTransform(*root).value_or(vve::Transform{});
local.translation.value.x += 2;
auto moved = render.setSceneInstanceNodeTransform(*car, *root, local);
if (!moved) return 11;
```

keep_hierarchy keeps the model's node tree in the instance. Setting a node's local transform moves every part below it, so moving the root moves the whole model. Use this instead of the plain instantiateScene call above. hierarchy_car.exe drives a car whose body and wheels follow their nodes.

Play a looping song

```
auto audio = engine.world().get<vve::AudioSystem>();
if (!audio.init()) return 7;
auto song = audio.loadSound(musicPath,
    vve::AudioLoadMode::on_demand);
if (!song) return 8;
auto voice = audio.play(*song,
    vve::AudioPlaybackOptions{.loop = true});
if (!voice) return 9;
```

Keep the voice for pause/resume/stop. setMasterVolume takes AudioVolume with gain 0..1. MP3 uses the manifest's mpg123 feature. Keep music, credits and DLLs with the executable. The crate game includes all three original V2 songs.

Sources: B, D, T, E, S, H | source key on page 8

8. When something goes wrong

Use the first failing stage and the source key.

Symptom	First fix to try
Tools or vcpkg missing	Install the C++ workload and set VCPKG_ROOT; reopen the shell.
import std / module failure	Use Ninja and matching MSVC/CMake; start with the build script.
No compatible Vulkan device	Update loader/driver; inspect required GPU features.
Model/texture missing	Check paths and working directory; print errorName.
Object missing/clipped	Instantiate it, check camera/light, and fit Camera::clip to the level.
Music missing or silent	Call audio.init; inspect lastError and file/DLL paths; check volume.
Debugger not configured	Install C/C++; choose Windows Debug (choose executable) in Run and Debug.

Where to read next

ID	Source file and starting line
R	README.md:39
C	src/CMakeLists.txt:8
E	examples/game/CMakeLists.txt:1
A	src/Engine.ixx:1
W	src/World.ixx:1
T	src/Types.ixx:1
B	src/Assets.ixx:1
D	src/RenderSystem.ixx:4
G	src/Window.ixx:1
S	src/Audio.ixx:1
H	examples/hierarchy_car/hierarchy_car.cpp:50

Edition: 4 October 2026, checkout 8123806a70ec. Both complete starters compile against current MSVC Debug modules. Windows Debug and Release engine builds passed 63/63 tests, including music startup. Focused snippets are excerpts. Use the full manual for internals.
Update, 15 October 2026 (commit 4b54ba1a): pages 2 and 7 add the scene hierarchy and hierarchy_car. The new snippet was compile-checked with Clang; the test count above is from the 4 October edition.

Sources: R, C, A | source key on page 8